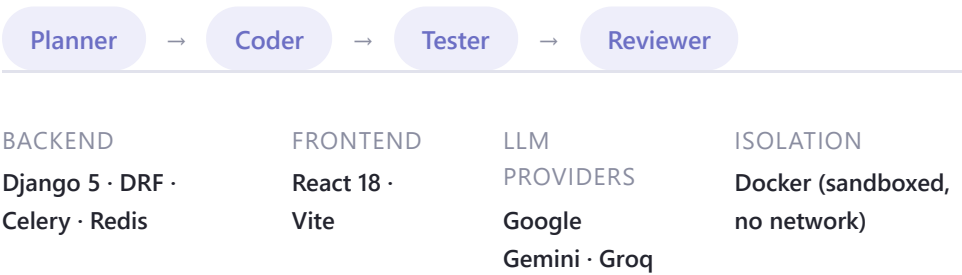


PROJECT DOCUMENTATION

Autonomous Coding Agent Swarm

A multi-agent orchestration system that plans, writes, tests, and reviews a real code change end-to-end — with every decision logged and visible.



1. What This Project Does

Agent Swarm automates a coding task the way a small engineering team would divide it up — except every "team member" is a specialized agent, and the whole exchange is logged. You give it a plain-English task description and the path to a local git repository. From there, four cooperating agents take over:

- **Planner** reads the task and the repo's file tree, and produces an ordered, concrete implementation plan.
- **Coder** follows that plan and rewrites the files it calls for, directly in your repo.
- **Tester** runs your *real* test suite inside an isolated, network-disabled Docker container and reports pass/fail — no LLM involved, because testing should be deterministic.
- **Reviewer** gives the final diff a fast sanity check against the plan before the task is marked done.

If the tests fail, the Coder gets another shot — with the actual failure output as context — up to a bounded number of retries, instead of looping forever or silently giving up. A React dashboard polls the backend and renders all of this as a live timeline (the **Agent Trace**), plus a diff viewer for exactly what the Coder changed.

Why this exists

This is a portfolio project built to demonstrate multi-agent orchestration end-to-end: a real Planner → Coder → Tester → Reviewer loop, backed by an async task pipeline, sandboxed execution, and observability by default — rather than a single LLM call dressed up as "agentic."

2. Tech Stack

Layer	Technology	Role
Backend / API	Django 5 + Django REST Framework	Task/AgentRun models, REST endpoints, free admin audit-trail UI
Async pipeline	Celery + Redis	Runs the Planner→Coder→Tester→Reviewer chain off the request thread; Redis is both the broker and result backend
Database	SQLite (dev)	Standard Django ORM only — no SQLite-only features — so it's a config change away from Postgres
Sandbox	Docker, via the <code>docker</code> Python SDK	One ephemeral, network-disabled container per Coder-write / Tester run
Planner LLM	Google Gemini (<code>google-genai</code> SDK)	Repo-wide reasoning and task decomposition into a step plan
Coder LLM	Google Gemini	Whole-file rewrites, given the plan + relevant file contents + prior test failures
Tester	pytest / npm test (no LLM)	Deterministic pass/fail inside the sandbox
Reviewer LLM	Groq (Llama 3.1 8B Instant)	Fast, bounded "does this diff match the plan" check
Frontend	React 18 + Vite	Task submission, live Agent Trace timeline, diff viewer
Diff rendering	react-diff-viewer-continued	Renders the Coder's before/after per file
Styling	Custom neumorphic CSS (light + dark)	Soft-UI theme driven entirely by CSS custom properties

3. Why This Tech Stack

Two LLM providers, on purpose

This is a deliberate design choice, not two APIs bolted on at random:

- **Planner & Coder** → **Gemini**. Both need to reason over repo-wide context — the file tree, the plan, prior file contents, and (on retry) the previous test failure. Gemini's context window handles that comfortably on the free tier.
- **Tester** → **no LLM at all**. Test results should be deterministic and reproducible. Programmatically parsing a real `pytest` exit code and output is strictly more trustworthy than asking a model to "judge" whether tests passed.
- **Reviewer** → **Groq**. A bounded, final "does this look reasonable" check doesn't need a large context window — it needs to be fast and cheap. Groq's low latency makes it a snappy last step rather than a bottleneck.

All provider calls go through one abstraction, `agents/llm_client.py`'s `call_llm(provider, messages)`, so swapping which provider any agent uses is a one-line change — not a rewrite. It also centralizes the free-tier handling both providers need: exponential backoff with jitter on 429s, capped at 3 retries.

Django + DRF + Celery + Redis

This combination gets a lot for free that a leaner stack would have to build by hand. The Django admin doubles as a zero-extra-frontend audit trail for every `Task` and `AgentRun` — genuinely useful for debugging a multi-step agent pipeline, not just a scaffold artifact. Celery is the standard way to move a multi-minute, multi-API-call pipeline off the HTTP request/response cycle, and Redis serving as both the broker and result backend keeps the infrastructure to a single extra service in dev.

Docker sandbox, not a subprocess

Executing LLM-generated code is the one place in this project where isolation is non-negotiable. Every Coder-written file is exercised inside a throwaway container with `network_disabled=True`, a memory cap, and an enforced timeout — the generated code cannot reach the network, exhaust the host, or hang the pipeline. A plain subprocess call would not give any of those guarantees.

Whole-file rewrites, not LLM-generated diffs

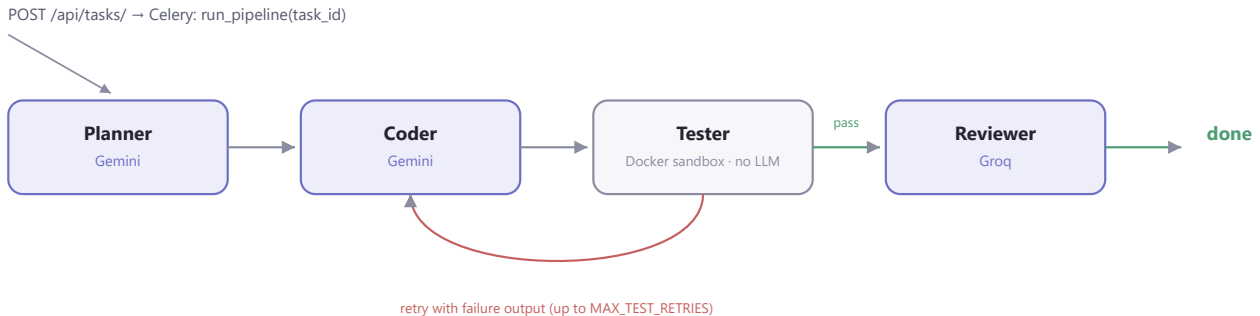
The Coder agent outputs strict JSON — `{"files": [{"path", "content"}]}` — with the complete new content of each file it touches, rather than a unified diff. LLM-generated unified diffs are fragile to parse: a single off-by-one context line breaks `git apply` outright. Whole-file rewrites are trivial to validate and apply safely (including a path-traversal check before anything touches disk). A real `git diff` is then computed *after* the files are written, so the Reviewer prompt and the frontend's diff viewer still see a proper diff — the model just never has to produce one directly.

React + Vite, polling first

The dashboard's job is to make an asynchronous, multi-minute pipeline legible in real time. Plain polling (every 2–3 seconds) gets that job done with a fraction of the complexity of a WebSocket layer, and it's what actually shipped for v1 — Django Channels remains a clearly-scoped stretch goal rather than a requirement the demo depends on.

4. How It Works

Pipeline flow



Step by step

- 1 Submit**
 - A task (description + local repo path) is created via `POST /api/tasks/` and enqueued as a Celery job.
- 2 Plan**
 - The Planner reads the task description and a trimmed repo file tree, and returns strict JSON: an ordered list of concrete implementation steps.
- 3 Code**
 - The Coder selects the files the plan actually names (or falls back to a size-capped sweep), sends their contents plus the plan to Gemini, and gets back whole-file rewrites. Those are written straight into the repo; a real `git diff` is computed immediately after.
- 4 Test**
 - The modified repo is bind-mounted into a fresh, network-disabled Docker container and its real test suite runs (`pytest` by default). Exit code and output are parsed — no model involved.
- 5 Retry if needed**
 - On failure, the Coder runs again with the actual test output as context, up to `MAX_TEST_RETRIES` (default 3) attempts, before the task is marked `failed`.
- 6 Review**
 - Once tests pass, the Reviewer checks the final diff against the plan and returns `{"approved", "concerns"}`.
- 7 Done**
 - The task is marked `done`. Every stage's full input/output — including the raw LLM response — is already sitting in the database.

Observability by default

Every agent invocation creates an `AgentRun` row *before* it calls the LLM or the sandbox (status `pending`), and updates it afterward — success or exception — so a crashed agent is always visible in the trace, never silently missing. Every stage's raw LLM response is persisted to `AgentRun.output`, so a failure can be diagnosed from the database without re-calling the API. All of this is visible two ways: the React dashboard's live Agent Trace, and the Django admin (`/admin/`), which needed zero extra frontend work to become a full audit log.

Data model

Model	Key fields
Task	<code>description</code> , <code>repo_path</code> , <code>status</code> (pending → planning → coding → testing → review → done/failed)
AgentRun	<code>task</code> (FK), <code>agent_type</code> , <code>provider</code> , <code>input_context</code> , <code>output</code> , <code>status</code> , <code>retry_count</code> , <code>duration_ms</code>

5. How to Start and Run It

Prerequisites

- Python 3.11+, Node 18+, Git
- **Docker Desktop**, running — required for the sandbox and for Redis via compose
- A free [Google AI Studio](#) API key (Gemini) and a free [Grog](#) API key

Setup

1 Start Redis

```
docker compose up -d
```

2 Build the sandbox image (once)

— `python:3.11-slim` alone has no test runner, and the sandbox has no network access at run time to install one.

```
cd backend
docker build -t agent-swarm-sandbox:latest -f docker/sandbox.Dockerfile .
```

3 Backend — install & migrate

```
cd backend
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env      REM then fill in GOOGLE_API_KEY / GROQ_API_KEY
python manage.py migrate
python manage.py test_llm   REM optional: confirms both API keys work
```

4 Backend — run the API server

(terminal 1)

```
python manage.py runserver
```

5 Backend — run the Celery worker

(terminal 2 — Windows needs `-P solo`)

```
celery -A orchestrator worker --loglevel=info -P solo
```

6 Frontend

(terminal 3)

```
cd frontend
npm install
npm run dev
```

Open the URL Vite prints — by default `http://localhost:5173` .

Try it — the built-in demo task

A ready-made target repo ships in `demo_repo/` : a tiny Flask app whose `/signup` endpoint has no validation, with two tests that fail against it. From the dashboard, submit:

Description:

Add input validation to the `/signup` endpoint: reject requests with an invalid email format or a missing password with a 400 response.

Repo path:

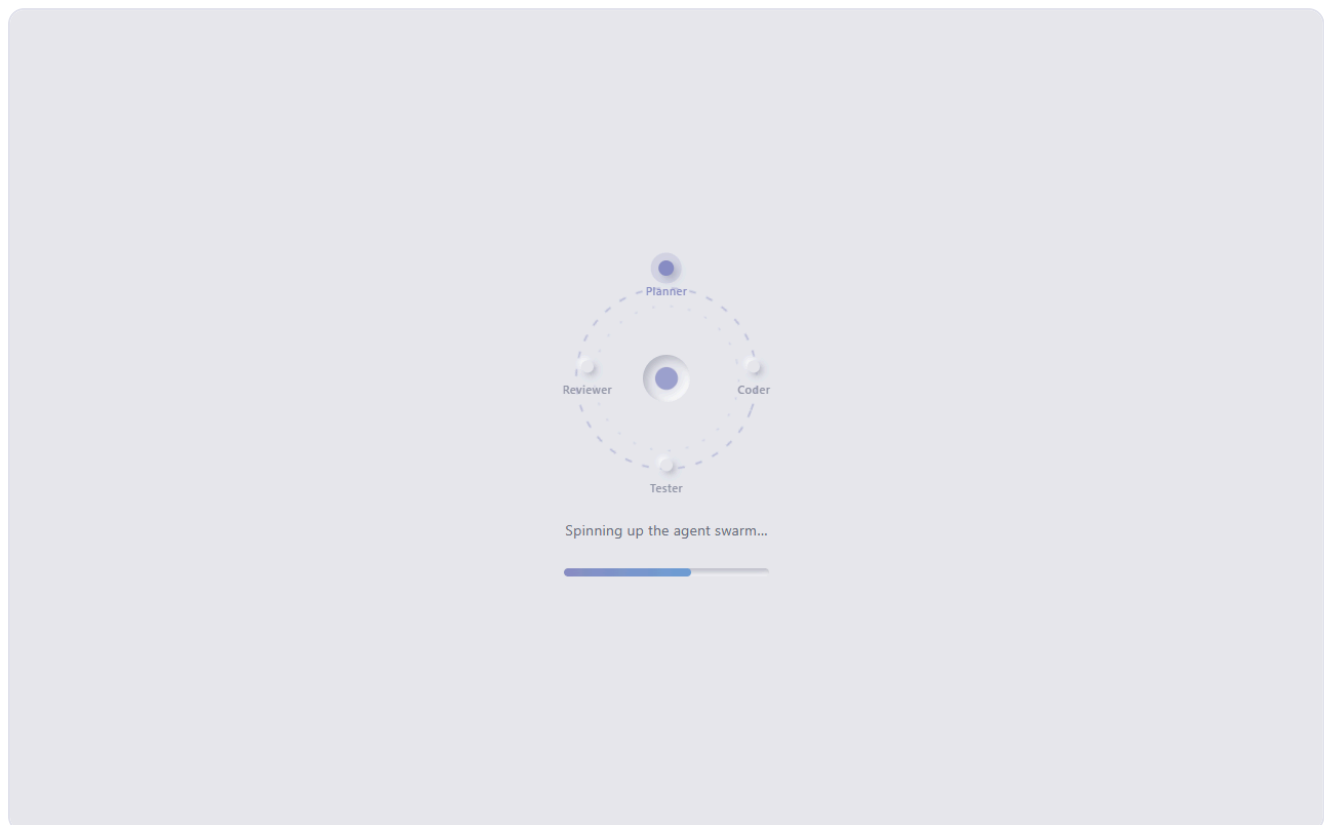
```
C:\intelFPGA\18.1\Practice\agent_swarm\demo_repo
```

Watch the Agent Trace go Planner → Coder → Tester → Reviewer and land on `done` . To run it again afterward, reset the repo first so the tests are failing again: `cd demo_repo && git checkout -- .`

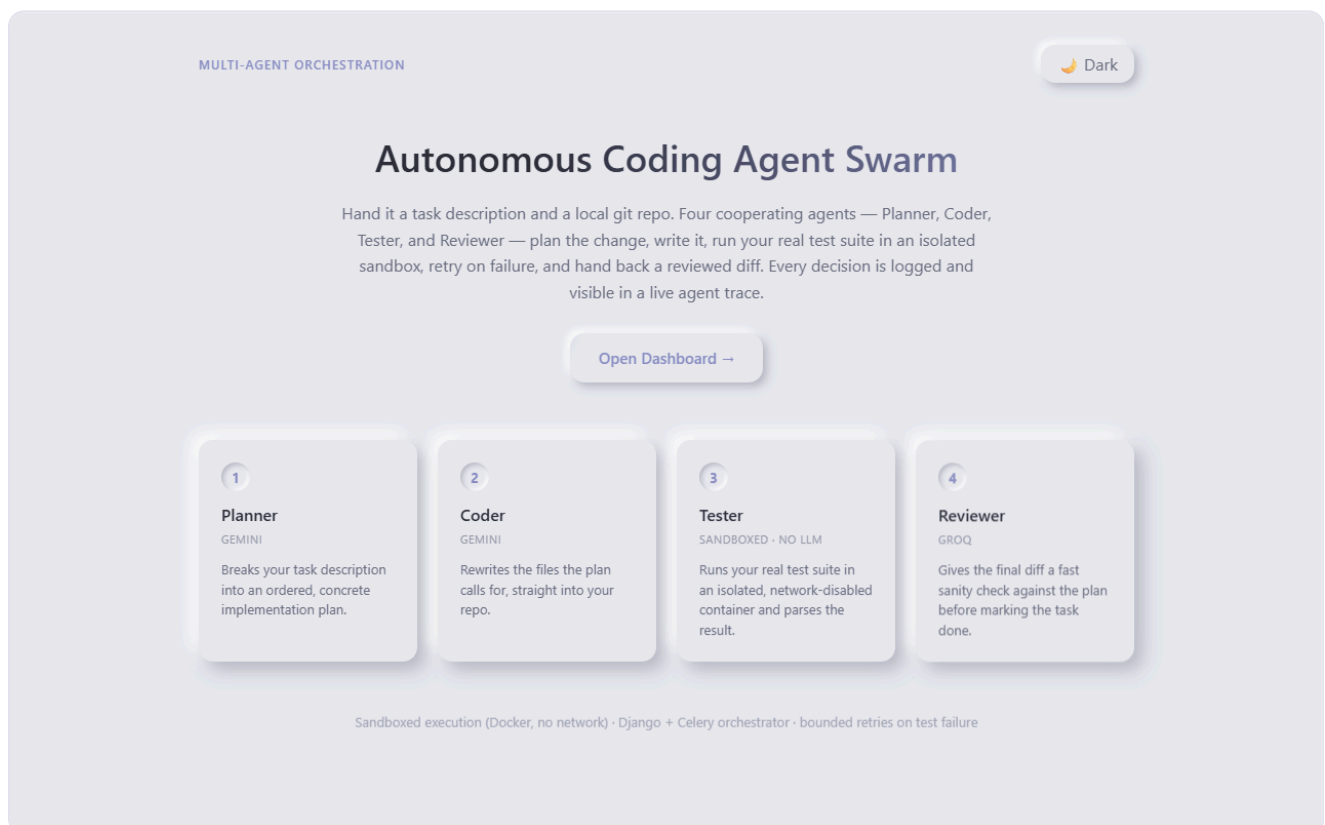
Troubleshooting notes from real setup runs

- **'docker' is not recognized in PowerShell** — a per-user Docker Desktop install doesn't always add itself to PATH. Use the full path once (`C:\Users\<you>\AppData\Local\Programs\DockerDesktop\resources\bin\docker.exe`), or add that folder to your user `PATH` permanently and reopen the terminal.
- **"Failed to create task" with no POST logged on the backend** — usually a leftover `npm run dev` process holding port 5173, pushing your actual tab onto 5174/5175, which the backend's `CORS_ALLOWED_ORIGINS` doesn't trust. Close stray dev-server terminals, or widen the allow-list in `backend/.env` .
- **Backend picks up `.env` only at startup** — restart `runserver` after editing it.

6. Screenshots



Splash screen shown once on load — four agent nodes cycling around a pulsing core before the landing page fades in.



Landing page — explains the pipeline (Planner → Coder → Tester → Reviewer) and links into the dashboard.

← Home

🌙 Dark

New Task

Description

e.g. Add input validation to the /signup endpoint

Repo path

C:\path\to\local\git\repo

Run Agent Swarm

Tasks

#5 • DONE
Add input validation to the /signup endpoint: reject requests with an invalid email format or a missing password with a 400 response.
C:\intelFPGA\18.1\Practice\agent_swarm\demo_repo

#4 • DONE
Add input validation to the /signup endpoint: reject requests with an invalid email format or a missing password with a 400 response.
C:\intelFPGA\18.1\Practice\agent_swarm\demo_repo

Agent Trace — Task #5

Planner via gemini21750 ms• SUCCESS

Coder via gemini14156 ms• SUCCESS

Tester1078 ms• SUCCESS

Reviewer via groq186 ms• SUCCESS

Diff

test_app.py

before

		Expand 22 lines ...
23	23	
24	24	def test_signup_rejects_missing_password(client):
25	25	resp = client.post("/signup", json={"email": "person@example.com"})
26	+	assert resp.status_code == 400
27	+	
28	+	
29	+	def test_signup_rejects_empty_password(client):
30	+	resp = client.post("/signup", json={"email": "person@example.com",
		"password": ""})

Dashboard after a completed run — task list on the left, live Agent Trace and the Coder's diff on the right.

7. Non-Goals (v1)

- No multi-repo support — one `repo_path` per task
- No authentication / user accounts — single-user local tool
- No LLM-based test generation — the Tester only runs a repo's existing tests
- No code execution outside the sandboxed container, ever

Agent Swarm — Autonomous Coding Agent Orchestrator